

Pattern Composite

GUITTON Valentin, FLAQUIERE Célian, LAUNAY Axel



C'est quoi un design pattern ?

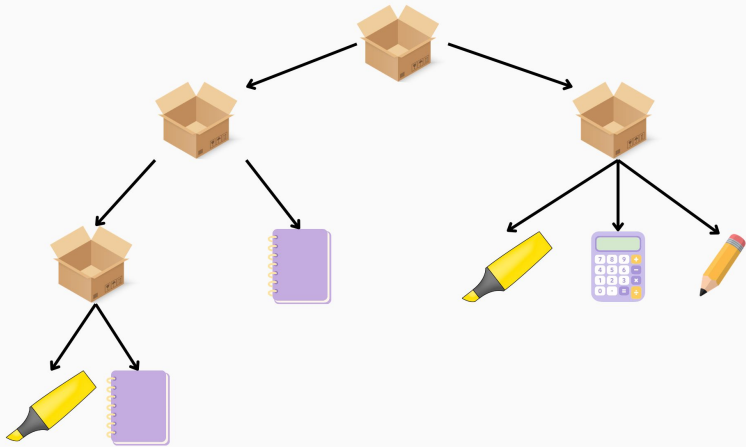
Patron de conception

Donne une forme prédéfinie de code / un type de besoin récurrent.

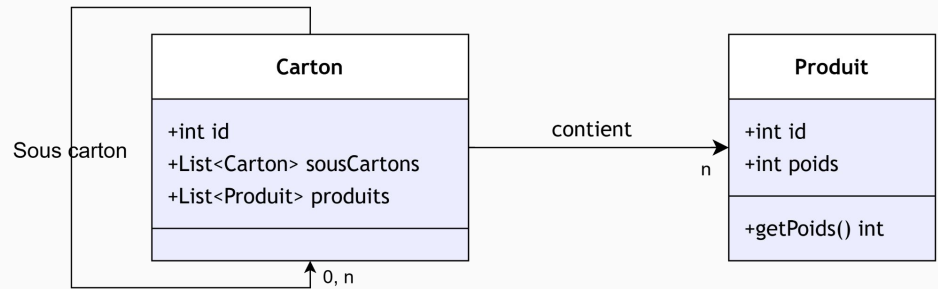
GoF : Erich Gamma, Richard Helm,
Ralph Johnson et John Vlissides

- Structurel
- Comportemental
- Créationnel

Problématique : Comment connaître le poids de ses colis ?



Solution



Le code associé

```
public class Carton { 2 usages
    public int id; 1 usage
    public List<Carton> cartons; 1 usage
    public List<Produit> produits; 1 usage

    public Carton(int id) { no usages
        this.id = id;
        this.cartons = new ArrayList<Carton>();
        this.produits = new ArrayList<Produit>();
    }
}
```

```
public class Produit { 2 usages
    public int id; 1 usage
    public int poids; 2 usages

    public Produit(int id, int poids) { no usages
        this.id = id;
        this.poids = poids;
    }

    public int getPoids() { no usages
        return poids;
    }
}
```

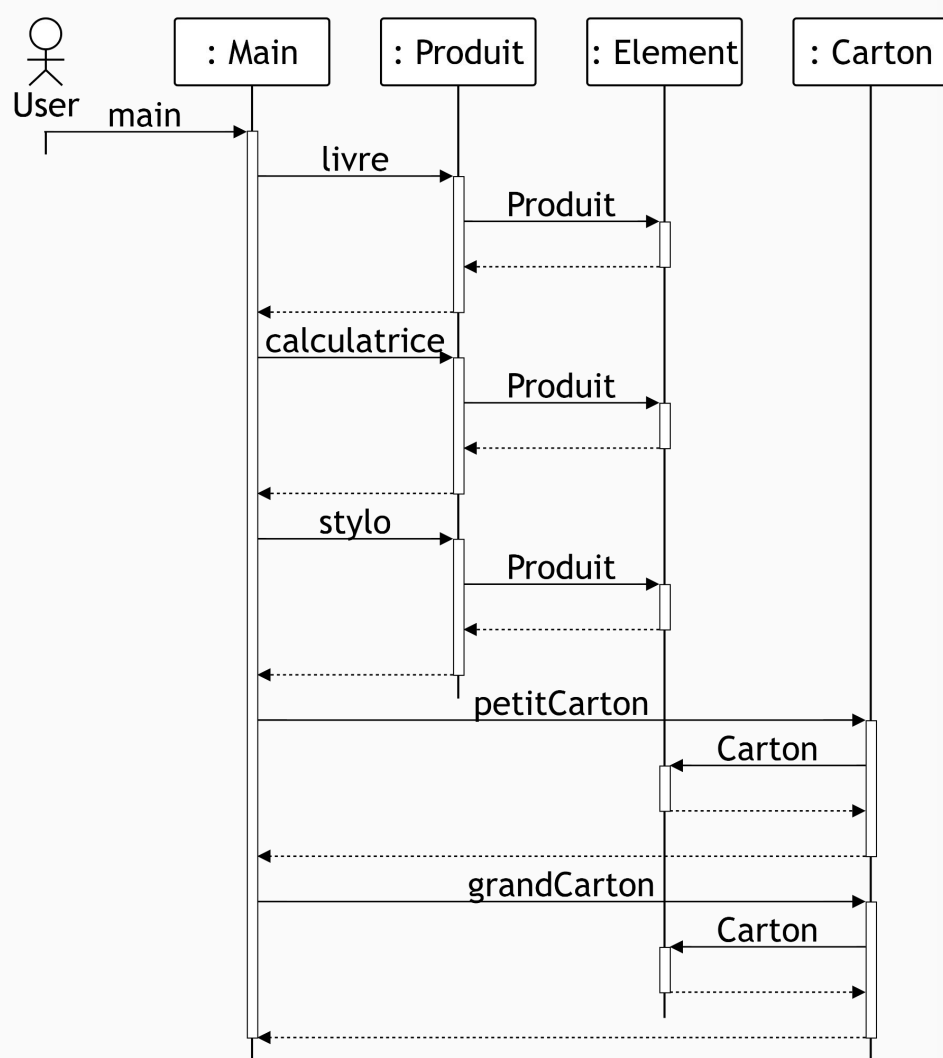
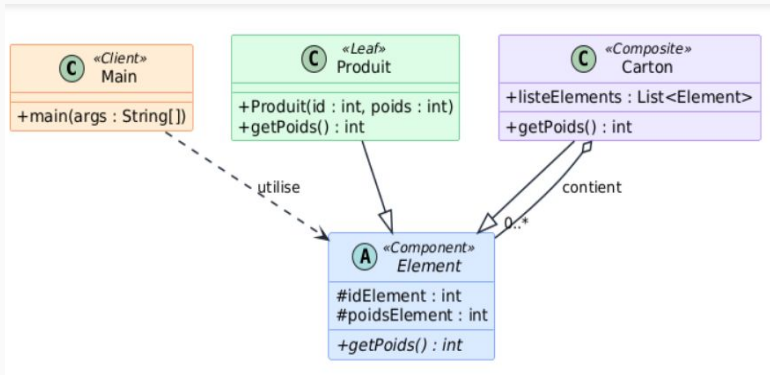


Diagramme de Séquence

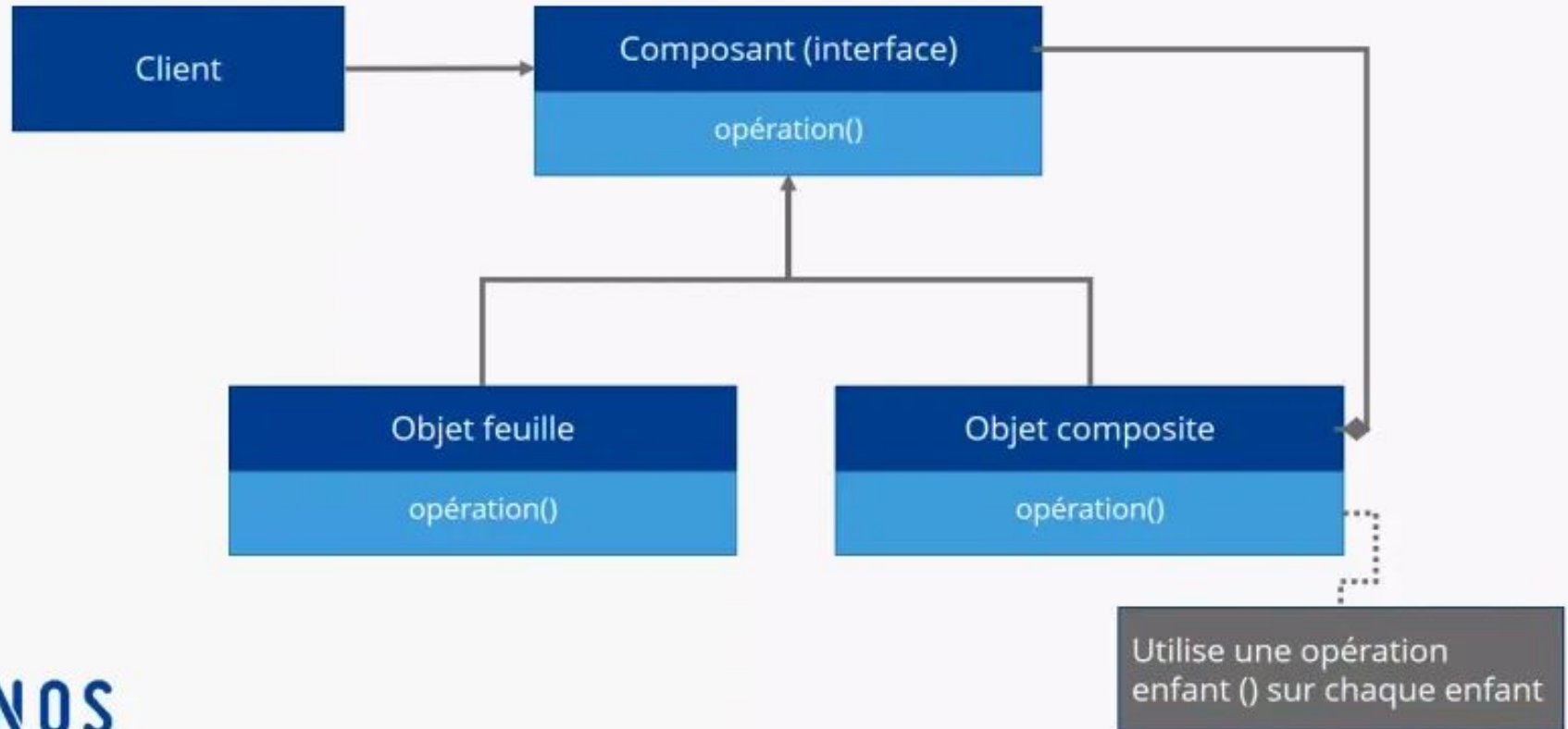
C'est alors qu'on va
utiliser le Pattern
Composite

Les 4 rôles :



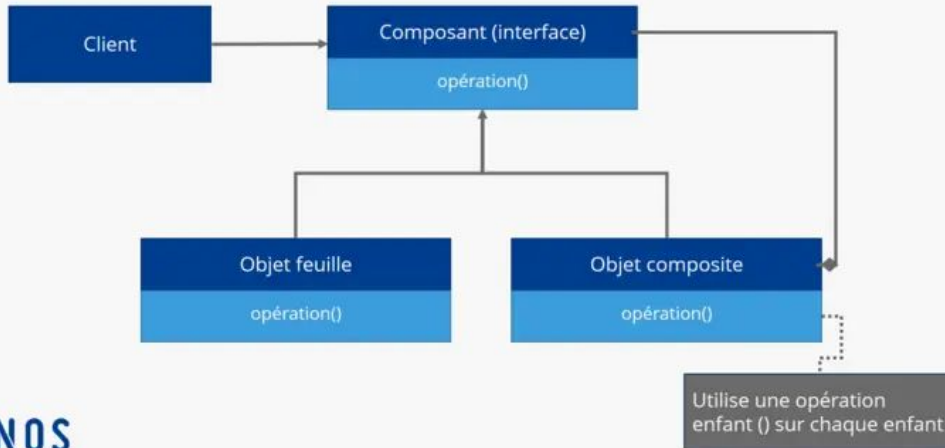
Rôle (GoF)	Dans notre exemple	En une phrase
Component	Élément	Le contrat commun : getPoids()
Leaf (feuille)	Produit	Élément terminal, il renvoie son propre poids
Composite	Carton	Contient des Élément, délègue à ses enfants
Client	Main	Utilise tout via Élément, sans faire de différence

Diagramme UML : Pattern Composite

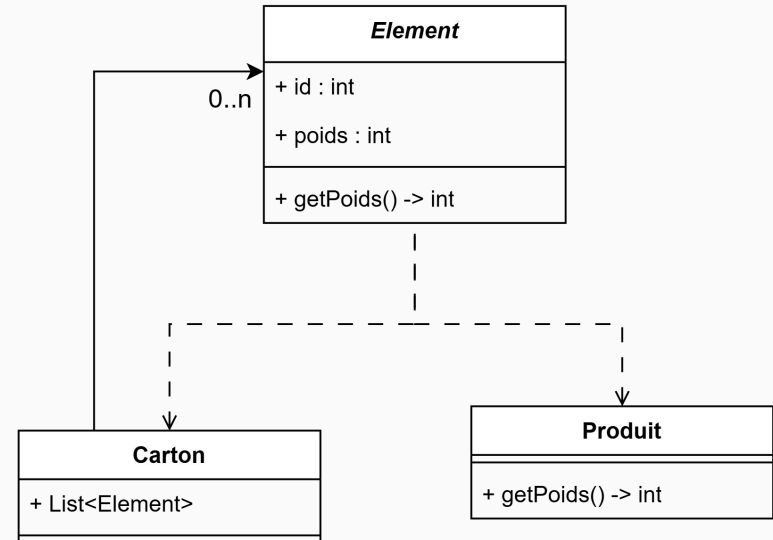


La solution :

Diagramme UML : modèle de composite pattern



Véritable solution



```

public class Produit extends Element { no usages
    // .....

    public Produit(int idElement, int poidsElement) { no usages
        super(idElement, poidsElement);
    }

    @Override 1 usage
    public int getPoids() {
        return poidsElement;
    }
}

public abstract class Element { 5 usages 2 inheritors
    protected int idElement; 2 usages
    protected int poidsElement; 2 usages

    public Element(int idElement, int poidsElement) { 3 usages
        this.idElement = idElement;
        this.poidsElement = poidsElement;
    }

    public int getIdElement() { no usages
        return idElement;
    }

    public abstract int getPoids(); 1 usage 2 implementations
}

```

Le code associé

```

public class Carton extends Element { no usages
    public List<Element> listeElements; 3 usages
    // .....

    public Carton(int idElement) { no usages
        super(idElement, poidsElement: 0);
        this.listeElements = new ArrayList<>();
    }

    public Carton(int idElement, List<Element> listeElements) { no usages
        super(idElement, poidsElement: 0);
        this.listeElements = listeElements;
    }

    @Override 1 usage
    public int getPoids() {
        int poidsTotal = 0;
        for (Element element : listeElements) {
            poidsTotal += element.getPoids();
        }
        return poidsTotal;
    }
}

```

SOLID

O : Open/Closed

On peut instancier des éléments sans toucher au composite

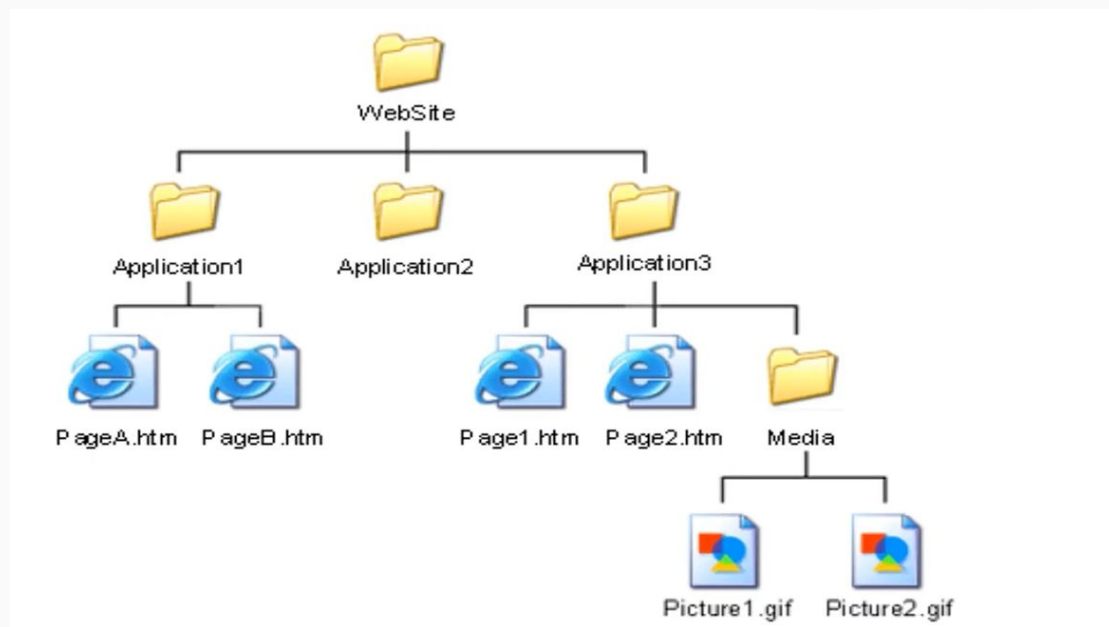
L : Liskov

Un composite peut remplacer l'autre là où on attend un Component. Liskov est violé si les feuilles lancent des exceptions.

D : Dependency inversion

Le composite et le client dépendent de l'abstraction, jamais des classes concrètes

Exemple 2



Exemple 3 : live coding

Exemple tiré du jeu vidéo League of Legends :

Sitographie

https://fr.wikipedia.org/wiki/Patron_de_conception

<https://refactoring.guru/fr/design-patterns/composite>

<https://www.ionos.fr/digitalguide/sites-internet/developpement-web/composite-pattern/>

[https://fr.wikipedia.org/wiki/Composite_\(patron_de_conception\)](https://fr.wikipedia.org/wiki/Composite_(patron_de_conception))